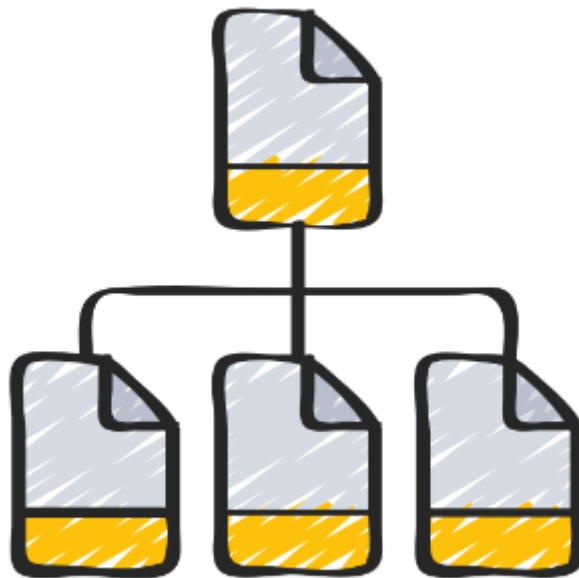


Sistemas operativos avanzados

Sistema de ficheros



Índice

1. Desarrollo del proyecto.....	1
1.1 Diseño y detalles de implementación de cada fase.....	1
1.1.1 Fase 1: Recuperación de metadatos.....	1
1.1.2 Fase 2: Mostrar directorios y archivos.....	4
1.1.3 Fase 3: Mostrar contenido de archivos FAT16	7
1.2 Estructuras de datos usadas y justificación	8
1.3 Escenarios de prueba y resultados para ambos sistemas de ficheros	12
1.3.1 Prueba de --info en EXT2	12
1.3.2 Prueba de --info en FAT16.....	13
1.3.3 Prueba de --tree en EXT2	13
1.3.4 Prueba de --tree en FAT16	13
1.3.5 Prueba con otra imagen EXT2	14
1.3.6 Prueba de --cat en FAT16	14
1.3.7 Pruebas de errores básicos	15
2. Estimación de tiempo dedicado a cada fase	16
3. Problemas encontrados y su resolución.....	16
4. Conclusiones.....	17

1. Desarrollo del proyecto

En este apartado se explica un poco cómo se ha desarrollado el proyecto. El programa está hecho en C y se llama fsutils. La idea principal era poder leer imágenes de sistemas de ficheros EXT2 y FAT16 directamente, sin tener que montarlas en Linux.

Para leer la información de las imágenes se han usado funciones de bajo nivel como open, read, lseek y close. Básicamente, open sirve para abrir la imagen, lseek para moverse a una posición concreta dentro del archivo, read para leer los datos que necesitamos y close para cerrar el archivo al final.

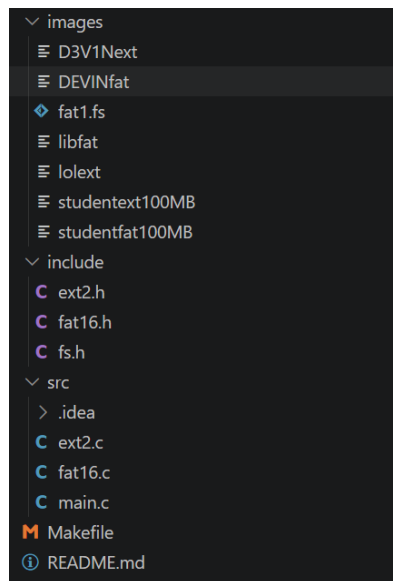


Figura 0. Estructura del programa

El proyecto está dividido en varios archivos para tener el código un poco más ordenado. En main.c se miran los argumentos que introduce el usuario por consola y, según la opción que se use, se llama a una función u otra. La parte de EXT2 está en ext2.c y la parte de FAT16 está en fat16.c. Las opciones principales que se han implementado son --info, --tree y --cat.

1.1 Diseño y detalles de implementación de cada fase

1.1.1 Fase 1: Recuperación de metadatos

En la primera fase se implementó la opción --info. Esta opción sirve para mostrar información básica de una imagen de sistema de ficheros.

El comando que se usa es:

```
Usage: ./fsutils --info <file_system>
```

Lo primero que hace el programa es abrir la imagen que recibe por parámetro. Después intenta detectar si esa imagen es EXT2 o FAT16. Según el tipo de sistema de ficheros, llama a las funciones correspondientes para imprimir la información.

```

if (strcmp(argv[1], "--info") == 0) {
    if (argc != 3) {
        printf("Usage: ./fsutils --info <file_system>\n");
        close(fd);
        return FS_ERROR;
    }

    if (is_ext2(fd)) {
        print_ext2_info(fd);
    } else if (is_fat16(fd)) {
        print_fat16_info(fd);
    } else {
        printf("Error: unsupported file system\n");
        close(fd);
        return FS_ERROR;
    }
}

```

Figura 1. Gestión de la opción --info y selección entre EXT2 y FAT16.

En el caso de EXT2, la detección se hace leyendo el magic number del superbloque. Este valor está en la posición 1024 + 56 de la imagen. Si el valor leído es 0xEF53, el programa entiende que la imagen es EXT2.

```

int is_ext2(int fd)
{
    uint16_t magic;

    if (lseek(fd, 1024 + 56, SEEK_SET) == -1) {
        return 0;
    }

    if (read(fd, &magic, sizeof(magic)) != (ssize_t) sizeof(magic)) {
        return 0;
    }

    if (magic == 0xEF53) {
        return 1;
    }

    return 0;
}

```

Figura 2. Detección de EXT2 leyendo el magic number del superbloque.

Una vez detectado EXT2, se lee el superbloque desde el byte 1024. De ahí se sacan los datos principales que se imprimen por pantalla, como el número de inodos, el número de bloques, el tamaño de bloque, los bloques libres, los inodos libres, el primer inodo, los inodos por grupo y el nombre del volumen.

```

void print_ext2_info(int fd)
{
    ext2_superblock sb;
    uint32_t tamBloque;
    char nombreVolumen[17];
    char ultimaRevision[40];
    char ultimoMontaje[40];
    char ultimaEscritura[40];

    if (lseek(fd, 1024, SEEK_SET) == -1) {
        printf("Error: cannot read EXT2 superblock\n");
        return;
    }

    if (read(fd, &sb, sizeof(ext2_superblock)) != (ssize_t) sizeof(ext2_superblock)) {
        printf("Error: cannot read EXT2 superblock\n");
        return;
    }

    tamBloque = 1024 << sb.s_log_block_size;
}

```

Figura 3. Lectura del superbloque EXT2 y cálculo del tamaño de bloque.

En el caso de FAT16, el programa lee el boot sector desde el inicio de la imagen. Para comprobar que realmente puede ser FAT16, se revisan varios campos básicos, como el tamaño del sector, los sectores por cluster, el número de FATs, los sectores reservados, las entradas del directorio raíz y los sectores por FAT. También se calcula el número de clusters para comprobar que está dentro del rango esperado para FAT16.

```
int is_fat16(int fd)
{
    fat16_boot_sector bs;
    uint32_t sectoresRoot;
    uint32_t sectoresTotales;
    uint32_t sectoresDatos;
    uint32_t numClusters;

    if (!leer_boot_fat16(fd, &bs)) {
        return 0;
    }

    if (bs.bytes_per_sector != 512 &&
        bs.bytes_per_sector != 1024 &&
        bs.bytes_per_sector != 2048 &&
        bs.bytes_per_sector != 4096) {
        return 0;
    }

    if (bs.sectors_per_cluster == 0) {
        return 0;
    }

    sectoresRoot = ((bs.root_entries * 32) + (bs.bytes_per_sector - 1)) / bs.bytes_per_sector;

    if (sectoresTotales <= bs.reserved_sectors + (bs.fat_count * bs.sectors_per_fat) + sectoresRoot) {
        return 0;
    }

    sectoresDatos = sectoresTotales - (bs.reserved_sectors + (bs.fat_count * bs.sectors_per_fat) + sectoresRoot);
    numClusters = sectoresDatos / bs.sectors_per_cluster;

    if (numClusters >= 4085 && numClusters < 65525) {
        return 1;
    }
}
```

Figura 4. Validaciones usadas para detectar una imagen FAT16.

Si la imagen se detecta como FAT16, se muestran datos como el nombre del sistema, el tamaño del sector, los sectores por cluster, los sectores reservados, el número de FATs, el número máximo de entradas del directorio raíz, los sectores por FAT y la etiqueta del volumen.

```
printf("----- Filesystem Information -----\n\n");
printf("Filesystem: FAT16\n\n");
printf("System name: %s\n", sistema);
printf("Sector size: %u\n", bs.bytes_per_sector);
printf("Sectors per cluster: %u\n", bs.sectors_per_cluster);
printf("Reserved sectors: %u\n", bs.reserved_sectors);
printf("# of FATs: %u\n", bs.fat_count);
printf("Max root entries: %u\n", bs.root_entries);
printf("Sectors per FAT: %u\n", bs.sectors_per_fat);
printf("Label: %s\n", etiqueta);
```

Figura 5. Información mostrada para sistemas FAT16.

En esta fase también se añadió un control básico de errores. Por ejemplo, si el usuario pone mal los argumentos, si no se puede abrir la imagen o si el sistema de ficheros no es soportado, el programa muestra un mensaje de error.

1.1.2 Fase 2: Mostrar directorios y archivos

En la segunda fase se implementó la opción `--tree`. Esta opción sirve para mostrar los archivos y carpetas de la imagen en forma de árbol, parecido al comando `tree` de Linux.

```
else if (strcmp(argv[1], "--tree") == 0) {
    if (argc != 3) {
        printf("Usage: ./fsutils --tree <file_system>\n");
        close(fd);
        return FS_ERROR;
    }

    if (is_ext2(fd)) {
        print_ext2_tree(fd);
    } else if (is_fat16(fd)) {
        print_fat16_tree(fd);
    } else {
        printf("Error: unsupported file system\n");
        close(fd);
        return FS_ERROR;
    }
}
```

Figura 6. Gestión de la opción `--tree` y selección entre EXT2 y FAT16.

El comando que se usa es:

```
Usage: ./fsutils --tree <file_system>
```

En EXT2, el recorrido empieza desde el inodo raíz, que es el inodo 2. Para poder leer un inodo, primero se calcula a qué grupo pertenece usando el número de inodo y la cantidad de inodos por grupo. Después se calcula la posición local de ese inodo dentro del grupo.

```
tamBloque = 1024 << sb->s_log_block_size;
grupo = (numInodo - 1) / sb->s_inodes_per_group;
posicion = (numInodo - 1) % sb->s_inodes_per_group;

if (!read_ext2_group_desc(fd, sb, &gd, grupo)) {
    return 0;
}

offsetInodo = (off_t) gd.bg_inode_table * tamBloque;
offsetInodo = offsetInodo + (off_t) posicion * sb->s_inode_size;
```

Figura 7. Cálculo del grupo, posición local y offset de un inodo EXT2.

Con el descriptor de grupo se puede saber dónde empieza la tabla de inodos. A partir de ahí se calcula el offset exacto del inodo dentro de la imagen y se lee su información.

Para saber si un inodo es un directorio, se comprueba el campo `i_mode`. Si es un directorio, se leen sus bloques de datos y esos bloques se interpretan como entradas de directorio.

```
static int is_ext2_directory(const ext2_inode *inode)
{
    if ((inode->i_mode & 0xF000) == 0x4000) {
        return 1;
    }

    return 0;
}
```

Figura 8. Comprobación de directorios EXT2

```
entry = (ext2_dir_entry *) (bufferBloque + pos);

if (entry->rec_len == 0) {
    break;
}
```

Figura 9. Lectura de entradas de directorio.

Al principio, esta parte solo leía los 12 bloques directos del inodo. Esto funcionaba para algunos casos simples, pero podía fallar si un directorio era más grande, porque se podían perder archivos. Por eso se añadió también la lectura de bloques indirectos. Ahora el programa tiene en cuenta los bloques directos, el indirecto simple, el indirecto doble y el indirecto triple.

```
for (i = 0; i < 12; i++) {
    leer_entradas_bloque_ext2(fd, inodoDir->i_block[i], tamBloque, lista);
}

leer_bloques_indirectos_ext2(fd, inodoDir->i_block[12], tamBloque, 1, lista);
leer_bloques_indirectos_ext2(fd, inodoDir->i_block[13], tamBloque, 2, lista);
leer_bloques_indirectos_ext2(fd, inodoDir->i_block[14], tamBloque, 3, lista);
```

Figura 10. Lectura de bloques directos e indirectos en EXT2.

También se mejoró la forma de imprimir el árbol. Antes se iba imprimiendo mientras se leía cada bloque, y eso podía hacer que el último elemento de un directorio no saliera bien marcado. Ahora primero se guardan las entradas válidas del directorio en una lista y después se imprimen. De esta forma se sabe mejor qué entrada es la última. También se descartan las entradas "." y "..", porque no interesa mostrarlas y podrían provocar recorridos incorrectos.

```
typedef struct {
    entradaValida *entradas;
    int contador;
    int capacidad;
} listaEntradas;
```

Figura 11. Uso de una lista dinámica para guardar entradas antes de imprimir el árbol.

En FAT16, el programa empieza leyendo el boot sector. Con esa información se calcula dónde está el directorio raíz y dónde empieza la zona de datos. El directorio raíz se lee directamente desde su posición dentro de la imagen.

```
static void imprimir_root_fat16(int fd, fat16_boot_sector *bs, int depth, int levels[])
{
    uint32_t zonaReservada;
    uint32_t zonaFat;
    uint32_t offsetRoot;
    uint32_t numEntradas;
    uint32_t i;
    fat16_dir_entry entrada;
    listaFat lista;

    zonaReservada = bs->reserved_sectors * bs->bytes_per_sector;
    zonaFat = bs->fat_count * bs->sectors_per_fat * bs->bytes_per_sector;
    offsetRoot = zonaReservada + zonaFat;
    numEntradas = bs->root_entries;
```

Figura 12. Cálculo de la posición del directorio raíz en FAT16.

Para entrar en subdirectorios se usa el cluster inicial que aparece en la entrada de directorio. Con ese cluster se calcula el offset dentro de la zona de datos. Si el directorio ocupa más de un cluster, se sigue la cadena de clusters consultando la FAT hasta llegar al final.

```
offsetCluster = inicioDatos + (off_t)(clusterActual - 2) * tamCluster;

if (!seek(fd, offsetCluster, SEEK_SET) == -1) {
    break;
}
```

```
if (!parar) {
    clusterActual = leer_entrada_fat(fd, bs, clusterActual);

    if (fin_cluster(clusterActual)) {
        break;
    }

    if (cluster_malo(clusterActual)) {
        break;
    }
}
```

Figura 13. Recorrido de subdirectorios FAT16 siguiendo la cadena de clusters.

En FAT16 no se recuperan los nombres largos LFN porque el enunciado indica que no hace falta. Por eso los nombres se muestran en formato corto 8.3, como por ejemplo SHOPPI~1.TXT o RECURS~1.C.

```
if (entrada->attr == 0x0F) {
    return 0;
}
```

```
memcpy(nombre, entrada->name, 8);
nombre[8] = '\0';

memcpy(extension, entrada->ext, 3);
extension[3] = '\0';
```

Figura 14. Creación de nombres 8.3 e ignorado de entradas LFN en FAT16.

También se añadieron comprobaciones para ignorar entradas eliminadas, entradas LFN, entradas de volumen y clusters no válidos.

1.1.3 Fase 3: Mostrar contenido de archivos FAT16

En la tercera fase se implementó la opción --cat.

```
else if (strcmp(argv[1], "--cat") == 0) {
    if (argc != 4) {
        printf("Usage: ./fsutils --cat <FAT16 file_system> <file>\n");
        close(fd);
        return FS_ERROR;
    }

    if (is_fat16(fd)) {
        print_fat16_cat(fd, argv[3]);
    } else {
        printf("Error: unsupported file system\n");
        close(fd);
        return FS_ERROR;
    }
}
```

Figura 15. Gestión de la opción --cat en main.c.

El comando usado es:

```
Usage: ./fsutils --cat <FAT16 file_system> <file>
```

Para buscar el archivo, el programa separa la ruta usando el carácter /. Primero busca la primera parte en el directorio raíz. Si la ruta tiene más carpetas, va entrando en cada subdirectorio usando su cluster inicial.

```
strncpy(ruta, nombreArchivo, sizeof(ruta) - 1);
ruta[sizeof(ruta) - 1] = '\0';

parte = strtok(ruta, "/");
if (parte == NULL) {
    printf("Error: file not found\n");
    return;
}

estoyEnRoot = 1;
clusterActual = 0;

while (parte != NULL) {
    siguiente = strtok(NULL, "/");

    if (estoyEnRoot) {
        if (!buscar_en_root(fd, &bs, parte, &entrada)) {
            printf("Error: file not found\n");
            return;
        }
    } else {
        if (!buscar_en_cluster(fd, &bs, clusterActual, parte, &entrada)) {
            printf("Error: file not found\n");
            return;
        }
    }
}
```

Figura 16. Separación de la ruta y búsqueda del archivo en FAT16.

Cuando llega a la última parte de la ruta, comprueba si es un archivo o un directorio. Si es un directorio, muestra un error. Si es un archivo, lee su contenido siguiendo la cadena de clusters en la FAT y lo imprime por pantalla.

```

while (cluster_valido(clusterActual) && bytesRestantes > 0) {
    off_t offsetCluster;
    uint32_t bytesLeer;

    offsetCluster = inicioDatos + (off_t)(clusterActual - 2) * tamCluster;

    if (lseek(fd, offsetCluster, SEEK_SET) == -1) {
        free(buffer);
        return;
    }

    if (read(fd, buffer, tamCluster) != (ssize_t) tamCluster) {
        free(buffer);
        return;
    }

    if (bytesRestantes < tamCluster) {
        bytesLeer = bytesRestantes;
    } else {
        bytesLeer = tamCluster;
    }

    write(1, buffer, bytesLeer);

    bytesRestantes = bytesRestantes - bytesLeer;
    clusterActual = leer_entrada_fat(fd, bs, clusterActual);
}

```

Figura 17. Lectura del contenido de un archivo FAT16 siguiendo la cadena de clusters.

Una mejora que se añadió fue usar `strcasecmp` en vez de `strcmp` al comparar nombres. Esto sirve para que funcione, aunque el usuario escriba la ruta en minúsculas y en FAT16 el nombre esté guardado en mayúsculas.

```

if (strcasecmp(nombreCompleto, nombreBuscado) == 0) {
    *resultado = entrada;
    return 1;
}

if (strcasecmp(nombreCompleto, nombreBuscado) == 0) {
    *resultado = *entrada;
    free(buffer);
    return 1;
}

```

Figura 18. Comparación de nombres FAT16 sin distinguir mayúsculas y minúsculas.

Por ejemplo, funcionan estas dos formas:

```

alex.oc@matagalls:~/SOA-FASE2> ./fsutils --cat images/DEVINfat COMPUS/EXTRA/TODOLIST.TXT
alex.oc@matagalls:~/SOA-FASE2> ./fsutils --cat images/DEVINfat compus/extra/todolist.txt

```

1.2 Estructuras de datos usadas y justificación

Para poder leer EXT2 y FAT16 se han creado varias estructuras en los archivos.h. Estas estructuras representan partes reales del sistema de ficheros, por eso es importante que coincidan con el formato que hay dentro de la imagen.

En EXT2 se usa la estructura `ext2_superblock` para guardar la información del superbloque. Esta estructura es importante porque contiene muchos datos básicos del sistema de ficheros, como el número de bloques, el número de inodos, el tamaño de bloque y otros valores que luego se usan para hacer cálculos.

```
typedef struct __attribute__((packed)) {
    uint32_t s_inodes_count;
    uint32_t s_blocks_count;
    uint32_t s_r_blocks_count;
    uint32_t s_free_blocks_count;
    uint32_t s_free_inodes_count;
    uint32_t s_first_data_block;
    uint32_t s_log_block_size;
    int32_t s_log_frag_size;
    uint32_t s_blocks_per_group;
    uint32_t s_frags_per_group;
    uint32_t s_inodes_per_group;
    uint32_t s_mtime;
    uint32_t s_wtime;
    uint16_t s_mnt_count;
    int16_t s_max_mnt_count;
    uint16_t s_magic;
    uint16_t s_state;
    uint16_t s_errors;
    uint16_t s_minor_rev_level;
    uint32_t s_lastcheck;
    uint32_t s_checkinterval;
    uint32_t s_creator_os;
    uint32_t s_rev_level;
    uint16_t s_def_resuid;
    uint16_t s_def_resgid;
    uint32_t s_first_ino;
    uint16_t s_inode_size;

```

```
    uint16_t s_block_group_nr;
    uint32_t s_feature_compat;
    uint32_t s_feature_incompat;
    uint32_t s_feature_ro_compat;
    uint8_t s_uuid[16];
    char s_volume_name[16];
    char s_last_mounted[64];
} ext2_superblock;
```

Figura 19. Estructura ext2_superblock, usada para guardar la información general del sistema EXT2.

También se usa ext2_group_desc, que representa un descriptor de grupo. Esta estructura sirve para encontrar la tabla de inodos de cada grupo. Como los inodos están organizados por grupos, no basta con saber el número de inodo, también hay que calcular su grupo y mirar dónde está la tabla de inodos correspondiente.

```
typedef struct __attribute__((packed)) {
    uint32_t bg_block_bitmap;
    uint32_t bg_inode_bitmap;
    uint32_t bg_inode_table;
    uint16_t bg_free_blocks_count;
    uint16_t bg_free_inodes_count;
    uint16_t bg_used_dirs_count;
    uint16_t bg_pad;
    uint8_t bg_reserved[12];
} ext2_group_desc;
```

Figura 20. Estructura ext2_group_desc, usada para localizar la tabla de inodos dentro de cada grupo EXT2.

La estructura ext2_inode representa un inodo. Se usa para saber si una entrada es un archivo o un directorio y para saber qué bloques pertenecen a ese archivo o directorio. El campo i_block es importante porque contiene los bloques directos e indirectos.

```
typedef struct __attribute__((packed)) {
    uint16_t i_mode;
    uint16_t i_uid;
    uint32_t i_size;
    uint32_t i_atime;
    uint32_t i_ctime;
    uint32_t i_mtime;
    uint32_t i_dtime;
    uint16_t i_gid;
    uint16_t i_links_count;
    uint32_t i_blocks;
    uint32_t i_flags;
    uint32_t i_osd1;
    uint32_t i_block[15];
    uint32_t i_generation;
    uint32_t i_file_acl;
    uint32_t i_dir_acl;
    uint32_t i_faddr;
    uint8_t i_osd2[12];
} ext2_inode;
```

Figura 21. Estructura ext2_inode, usada para representar un inodo y acceder a sus bloques directos e indirectos.

La estructura ext2_dir_entry se usa para leer las entradas de un directorio. Cada entrada contiene el número de inodo, el tamaño de la entrada y el nombre del archivo o carpeta.

```
✓ typedef struct __attribute__((packed)) {
    uint32_t inode;
    uint16_t rec_len;
    uint8_t name_len;
    uint8_t file_type;
    char name[];
} ext2_dir_entry;
```

Figura 22. Estructura ext2_dir_entry, usada para leer las entradas de un directorio EXT2.

Además, se creó una estructura auxiliar llamada listaEntradas. Esta lista se usa para guardar las entradas válidas de un directorio antes de imprimirlas. Se hizo así para poder saber cuál es el último elemento del directorio y para que el árbol salga mejor. También evita depender de un array fijo pequeño, porque usa malloc y realloc.

En FAT16 se usa la estructura fat16_boot_sector. Esta estructura guarda la información del boot sector, como el tamaño del sector, los sectores por cluster, el número de FATs, las entradas del directorio raíz y los sectores por FAT. Con estos datos se pueden calcular las posiciones importantes de la imagen.

```
typedef struct __attribute__((packed)) {
    uint8_t jump[3];
    char oem_name[8];
    uint16_t bytes_per_sector;
    uint8_t sectors_per_cluster;
    uint16_t reserved_sectors;
    uint8_t fat_count;
    uint16_t root_entries;
    uint16_t total_sectors_16;
    uint8_t media_descriptor;
    uint16_t sectors_per_fat;
    uint16_t sectors_per_track;
    uint16_t heads;
    uint32_t hidden_sectors;
    uint32_t total_sectors_32;
    uint8_t drive_number;
    uint8_t reserved1;
    uint8_t boot_signature;
    uint32_t volume_id;
    char volume_label[11];
    char fs_type[8];
} fat16_boot_sector;
```

Figura 23. Estructura fat16_boot_sector, usada para obtener la información principal del sistema FAT16.

También se usa fat16_dir_entry, que representa una entrada de directorio FAT16. Esta estructura contiene el nombre, la extensión, los atributos, el cluster inicial y el tamaño del archivo. Se usa tanto para mostrar el árbol como para buscar archivos con --cat.

```
typedef struct __attribute__((packed)) {
    char name[8];
    char ext[3];
    uint8_t attr;
    uint8_t reserved;
    uint8_t creation_time_tenths;
    uint16_t creation_time;
    uint16_t creation_date;
    uint16_t last_access_date;
    uint16_t first_cluster_high;
    uint16_t last_write_time;
    uint16_t last_write_date;
    uint16_t first_cluster_low;
    uint32_t file_size;
} fat16_dir_entry;
```

Figura 24. Estructura fat16_dir_entry, usada para leer archivos y directorios dentro de FAT16.

Para FAT16 también se creó una lista auxiliar llamada listaFat. Su función es parecida a la lista de EXT2: guardar las entradas válidas antes de imprimirlas. Así se pueden descartar antes las entradas que no interesan, como "." y "..", entradas eliminadas, entradas LFN o entradas de volumen.

Las estructuras están marcadas como packed para evitar que el compilador añada bytes extra entre campos. Esto es importante porque el programa lee directamente los datos desde la imagen, y si la estructura no coincide con el formato real, los valores podrían leerse mal.

1.3 Escenarios de prueba y resultados para ambos sistemas de ficheros

Durante el desarrollo se hicieron varias pruebas para comprobar que el programa funcionaba con EXT2 y con FAT16.

Primero se probó que el proyecto compilara correctamente:

```
alex.oc@matagalls:~/SOA-FASE2>make clean
rm -f *.o fsutils
alex.oc@matagalls:~/SOA-FASE2>make
gcc -Wall -Wextra -std=c11 -Iinclude -c src/main.c
gcc -Wall -Wextra -std=c11 -Iinclude -c src/ext2.c
gcc -Wall -Wextra -std=c11 -Iinclude -c src/fat16.c
gcc -Wall -Wextra -std=c11 -Iinclude -o fsutils main.o ext2.o fat16.o
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 25. Compilación del proyecto usando make clean y make.

El resultado fue correcto, ya que se generó el ejecutable fsutils sin errores.

1.3.1 Prueba de --info en EXT2

Comando usado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --info images/studentext100MB
```

Resultado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --info images/studentext100MB
----- Filesystem Information -----

Filesystem: EXT2

INODE INFO
Size: 256
Num Inodes: 25600
First Inode: 11
Inodes per group: 25600
Free Inodes: 25542

BLOCK INFO
Block size: 4096
Reserved blocks: 1280
Free blocks: 23933
Total blocks: 25600
First block: 0
Blocks per group: 32768
Fragments per group: 32768

VOLUME INFO
Volume name:
Last checked: Fri Mar 28 21:07:27 2025
Last mounted: Fri Mar 28 21:08:07 2025
Last written: Fri Mar 28 21:09:24 2025
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 26. Resultado de la opción --info con una imagen EXT2.

El programa detectó la imagen como EXT2 y mostró información de inodos, bloques y volumen. Con esta prueba se comprobó que la lectura del superbloque funcionaba correctamente.

1.3.2 Prueba de --info en FAT16

Comando usado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --info images/studentfat100MB
```

Resultado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --info images/studentfat100MB
----- Filesystem Information -----

Filesystem: FAT16

System name: mkfs.fat
Sector size: 512
Sectors per cluster: 4
Reserved sectors: 4
# of FATs: 2
Max root entries: 512
Sectors per FAT: 200
Label: NO NAME
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 27. Resultado de la opción --info con una imagen FAT16.

El programa detectó la imagen como FAT16 y mostró información como tamaño de sector, sectores por cluster, número de FATs, entradas máximas del directorio raíz y sectores por FAT. Con esto se comprobó que la lectura del boot sector funcionaba.

1.3.3 Prueba de --tree en EXT2

Comando usado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --tree images/studentext100MB
```

Resultado:

```

├── lost+found
├── ASO
│   ├── shopping_list.txt
│   └── pipes.sh
├── donut
│   ├── donut.c
│   ├── .donete
│   └── donete_pro.c
├── PAED
│   └── bogo.c
├── Prog
│   ├── sem1
│   │   └── iterative_fibonacci.c
│   ├── sem2
│   │   └── recursive_fibonacci.c
├── PrPrI
│   ├── main.c
│   ├── globals.h
│   └── .git
│       ├── branches
│       ├── hooks
│       │   ├── pre-rebase.sample
│       │   ├── pre-receive.sample
│       │   ├── push-to-checkout.sample
│       │   ├── pre-merge-commit.sample
│       │   ├── pre-push.sample
│       │   ├── prepare-commit-msg.sample
│       │   ├── pre-commit.sample
│       │   ├── commit-msg.sample
│       │   └── fsmonitor-watchman.sample
├── sendemail-validate.sample
├── applypatch-msg.sample
├── post-update.sample
├── update.sample
├── pre-applypatch.sample
├── info
│   ├── exclude
│   ├── description
│   ├── refs
│   │   ├── heads
│   │   └── tags
│   ├── HEAD
│   ├── objects
│   │   ├── pack
│   │   └── info
│   └── config
└── SO
    └── practica.c
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 28. Resultado de la opción --tree con una imagen EXT2.

1.3.4 Prueba de --tree en FAT16

Comando usado:

```
./fsutils --tree images/studentfat100MB
```

Resultado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --tree images/studentfat100MB
ASO
├── SHOPPI~1.TXT
├── PIPES.SH
├── DONUT
│   ├── DONUT.C
│   ├── DONETE~1
│   └── DONETE~1.C
├── PAED
│   └── BOGO.C
├── PROG
│   ├── SEM1
│   │   └── ITERAT~1.C
│   └── SEM2
│       └── RECURS~1.C
├── PRPRI
│   ├── MAIN.C
│   ├── GLOBALS.H
│   └── GIT~1
│       ├── BRANCHES
│       └── HOOKS
│           ├── PRE-RE~1.SAM
│           ├── PRE-RE~2.SAM
│           ├── PUSH-T~1.SAM
│           ├── PRE-ME~1.SAM
│           ├── PRE-PU~1.SAM
│           ├── PREPAR~1.SAM
│           ├── PRE-CO~1.SAM
│           └── COMMIT~1.SAM
└── SO
    └── PRACTICA.C
```

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --tree images/D3V1Next
COMMIT~1.SAM
FSMONI~1.SAM
SENDEM~1.SAM
APPLY~1.SAM
POST~U~1.SAM
UPDATE~1.SAM
PRE~AP~1.SAM
INFO
├── EXCLUDE
├── DESCR~1
├── REFS
│   ├── HEADS
│   └── TAGS
├── HEAD
├── OBJECTS
│   ├── PACK
│   └── INFO
├── CONFIG
└── SO
    └── PRACTICA.C
```

Figura 29. Resultado de la opción --tree con una imagen FAT16 usando nombres 8.3.

1.3.5 Prueba con otra imagen EXT2

Comando usado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --tree images/D3V1Next
```

Resultado:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --tree images/D3V1Next
lost+found
pwii
aos
├── I100VveeThissSUUBJjjectmmdir.txt
├── compilers
├── parser
├── lexer
├── tree
└── whatIsAFollow.txt
```

Figura 30. Prueba adicional de --tree con otra imagen EXT2.

1.3.6 Prueba de --cat en FAT16

Comando usado en mayúsculas:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --cat images/DEVINfat COMPUS/EXTRA/TODOLIST.TXT
TODO LIST:

1. START P1!!
2. BUY CHATGPT4 :/
3. CRY
```

Figura 31. Prueba de --cat en FAT16 usando la ruta en mayúsculas

Comando usado en minúsculas:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --cat images/DEVINfat compus/extra/todolist.txt
TODO LIST:

1. START P1!!
2. BUY CHATGPT4 :/
3. CRY
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 32. Prueba de --cat en FAT16 usando la ruta en minúsculas.

1.3.7 Pruebas de errores básicos

También se probaron algunos errores simples.

Si se ejecuta el programa sin argumentos:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils
Usage: ./fsutils --info <file_system>
Usage: ./fsutils --tree <file_system>
Usage: ./fsutils --cat <FAT16 file_system> <file>
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 33. Ejecución del programa sin argumentos, mostrando el uso correcto de fsutils.

Si se intenta abrir una imagen que no existe:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --info images/noexiste
Error: cannot open file system
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 34. Prueba de error al intentar abrir una imagen que no existe.

Si se intenta usar --cat con una imagen EXT2:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --cat images/studenttext100MB practica.c
Error: unsupported file system
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 35. Prueba de error al intentar usar --cat con una imagen EXT2.

Si se intenta abrir un archivo que no existe en FAT16:

```
alex.oc@matagalls:~/SOA-FASE2>./fsutils --cat images/DEVINfat compus/extra/noexiste.txt
Error: file not found
alex.oc@matagalls:~/SOA-FASE2>
```

Figura 36. Prueba de error al intentar abrir un archivo inexistente dentro de FAT16.

2. Estimación de tiempo dedicado a cada fase

La dedicación de tiempo del proyecto fue aproximada, ya que algunas fases necesitaron más trabajo después de recibir correcciones y probar más imágenes.

<i>Fase</i>	<i>Tiempo aproximado</i>
<i>Fase 1: Recuperación de metadatos</i>	4 horas
<i>Fase 2: Mostrar directorios y archivos</i>	8 horas
<i>Fase 3: Mostrar contenido de archivos FAT16</i>	6 horas
<i>Correcciones y pruebas</i>	5 horas
<i>README e informe</i>	2 horas

La fase que llevó más tiempo fue la fase 2, porque no solo había que mostrar el directorio raíz, sino también recorrer subdirectorios en EXT2 y FAT16. Además, después de la retroacción se tuvo que mejorar el recorrido de EXT2 para leer bloques indirectos y también corregir la forma de imprimir el árbol.

La fase 1 fue más directa, pero también necesitó una mejora en la detección de FAT16. La fase 3 fue más concreta, porque se centraba en buscar un archivo dentro de FAT16 y mostrar su contenido siguiendo la cadena de clusters.

3. Problemas encontrados y su resolución

Durante el proyecto fueron apareciendo varios problemas, sobre todo al probar con más imágenes y después de recibir la retroacción de las fases.

En la fase 1, uno de los problemas estaba en la detección de FAT16. Al principio se comprobaba principalmente la cadena "FAT16" en una posición concreta del boot sector, pero esto era una comprobación bastante débil. Podía fallar si la cadena no estaba justo ahí o incluso aceptar alguna imagen que no fuera realmente FAT16.

Para arreglarlo, se mejoró la función `is_fat16`. En vez de mirar solo esa cadena, ahora se comprueban varios campos del boot sector, como el tamaño del sector, los sectores por cluster, los sectores reservados, el número de FATs, las entradas del directorio raíz y los sectores por FAT. También se calcula el número de clusters para comprobar que entra dentro del rango de FAT16. Con esto la detección queda más segura.

En la fase 2, el problema principal estaba en EXT2. La primera versión solo leía los 12 bloques directos del inodo. Esto funcionaba con directorios pequeños, pero no era suficiente para directorios grandes. Por eso se añadió la lectura de bloques indirectos. Ahora se recorren los bloques directos, el indirecto simple, el indirecto doble y el indirecto triple.

También hubo que mejorar la forma de imprimir el árbol. Antes se imprimían las entradas mientras se leían, y eso podía hacer que el último elemento de un directorio saliera mal marcado. Además, las entradas "." y ".." podían afectar al resultado. La solución fue guardar

primero las entradas válidas en una lista y luego imprimirlas. Así se puede calcular mejor cuál es el último elemento.

Otro problema eran algunos límites fijos en arrays y buffers. Para evitar que el recorrido se cortara si había más entradas, se empezó a usar memoria dinámica con malloc y realloc en las listas de entradas.

En FAT16, el recorrido de clusters ya estaba planteado, pero se mejoró el control de casos especiales. Se añadieron comprobaciones para detectar el final de cadena, el bad cluster 0xFFFF y evitar seguir valores que no sean clusters normales. También se ignoran entradas eliminadas, entradas LFN y entradas de volumen.

En FAT16 los nombres largos LFN no se recuperan. Esto no se considera un error porque el enunciado indicaba que no era necesario. Por eso se muestran nombres en formato corto 8.3, como SHOPPI~1.TXT o RECURS~1.C.

En la fase 3 apareció otro problema con --cat. FAT16 guarda normalmente los nombres cortos en mayúsculas, así que si se escribía la ruta en minúsculas, el programa no encontraba el archivo. Para solucionarlo, se cambió la comparación de nombres usando strcasecmp en vez de strcmp. Así funciona tanto con mayúsculas como con minúsculas.

Por último, también hubo un problema de espacio al probar imágenes de 1GB en el entorno. No era un fallo del código, sino que no quedaba espacio suficiente para compilar o generar el ejecutable. Se solucionó eliminando las imágenes grandes que no eran necesarias y usando las imágenes pequeñas para las pruebas principales.

4. Conclusiones

En conclusión, con este proyecto se ha podido ver de forma más práctica cómo funcionan por dentro algunos sistemas de ficheros, en este caso EXT2 y FAT16. Al principio parecía que solo era leer datos de una imagen, pero luego se ve que hay que calcular bien offsets, tamaños, clusters, inodos y entradas de directorio para que todo salga correctamente.

La parte de EXT2 me ha servido para entender mejor la importancia del superbloque, los grupos y los inodos. También se ha visto que no basta con leer solo los bloques directos, porque en directorios más grandes pueden aparecer bloques indirectos y si no se tienen en cuenta se pierden archivos.

En FAT16, la parte más importante ha sido entender cómo se calcula la zona del directorio raíz y la zona de datos, y cómo se sigue la cadena de clusters usando la FAT. También se ha comprobado que los nombres en FAT16 pueden aparecer en formato corto 8.3, y que esto es normal porque en este proyecto no se pedía recuperar los nombres largos LFN.

Una de las partes más complicadas del proyecto ha sido el comando --tree, porque había que recorrer directorios y subdirectorios y además imprimirlos con un formato correcto. También ha sido importante corregir la forma de guardar las entradas antes de imprimirlas, para que el árbol no saliera mal con los últimos elementos.

El comando --cat también ha servido para reforzar la parte de FAT16, ya que no solo había que encontrar el archivo, sino también leer su contenido siguiendo los clusters. Además, se corrigió el problema de mayúsculas y minúsculas usando strcasecmp.